EXPECTATION–MAXIMIZATION ALGORITHM

A Project Report

Submitted in May 2026

Team Members

Chavi Makana

Anubhav Kumari

Kartik Manmode

Aryan Patel

Rajveer Singh

Department of Computer Science
Project Report
2026

Abstract

The Expectation–Maximization (EM) algorithm is a widely used iterative optimization technique for estimating parameters in statistical models that contain latent or hidden variables. In many real-world problems, the observed data is incomplete or generated from multiple hidden sources, making direct maximum likelihood estimation difficult. The EM algorithm solves this problem by dividing the optimization process into two iterative steps known as the Expectation step (E-step) and the Maximization step (M-step).

In the E-step, the algorithm estimates the expected values of hidden variables using the current parameter values. In the M-step, the model parameters are updated to maximize the expected log-likelihood obtained from the E-step. These two steps are repeated until convergence is achieved. The EM algorithm is based on the concept of likelihood maximization and Jensen's inequality, which ensures that the likelihood value improves after every iteration.

This report presents the theoretical background, derivation, implementation, and analysis of the EM algorithm. Important concepts such as latent variable models, conditional probability, likelihood functions, Gaussian distributions, and convergence are discussed in detail. Special emphasis is given to Gaussian Mixture Models (GMMs), where the EM algorithm is commonly used for probabilistic clustering and density estimation.

The practical implementation of the EM algorithm is carried out using Python programming language on synthetic datasets. Experimental analysis is performed to study convergence behavior, likelihood progression, clustering performance, and sensitivity to initialization. The report also discusses important limitations of the EM algorithm such as local maxima, slow convergence, and initialization dependence.

Overall, this project demonstrates how the EM algorithm provides an effective framework for solving incomplete-data optimization problems and highlights its importance in machine learning, statistics, and artificial intelligence applications.

Contents

1 Introduction 1

1.1 Overview of the EM Algorithm 1

1.2 Motivation and Need 2

1.3 Applications of EM 3

2 Mathematical Background 4

2.1 Probability and Conditional Probability 4

2.2 Likelihood and Log-Likelihood 5

2.3 Jensen's Inequality 6

3 Latent Variable Models 7

3.1 Introduction to Latent Variables 7

3.2 Observable vs Hidden Variables 8

3.3 Challenges in Direct Maximum Likelihood 9

4 Expectation–Maximization Algorithm 10

4.1 Introduction to EM Algorithm 10

4.2 The E-Step 11

4.3 The M-Step 12

4.4 Convergence of EM 13

4.5 Lower-Bound Interpretation using Jensen's Inequality 14

5 Gaussian Mixture Models 15

5.1 Introduction to Mixture Models 15

5.2 Gaussian Mixture Model Formulation 16

5.3 EM for GMM 17

6 Implementation and Results 18

6.1 Python Implementation 18

6.2 Experimental Results 19

6.3 Sensitivity to Initialization 20

7 Limitations and Applications 21

7.1 Limitations of EM 21

7.2 Real-World Applications 22

8 Conclusion 23

8.1 Summary and Future Scope 23

References 24

Books, Papers, and Online Sources 24

List of Figures

Figure 1.1 Flowchart of the Expectation-Maximization (EM) algorithm 6

Figure 2.1 ELBO lower bound tightening during successive EM iterations 11

Figure 3.1 Probabilistic graphical model representing latent-variable relationships 14

Figure 4.1 EM convergence behaviour on synthetic Gaussian mixture data 19

Figure 5.1 Gaussian Mixture Model density for varying number of components K 23

Figure 6.1 E-step responsibilities and soft cluster assignments for data points 24

Figure 7.1 EM-based Gaussian Mixture Model clustering on a synthetic dataset .. 28

Figure 8.1 Sensitivity of the EM algorithm to initialization and local optima 31

Chapter 1

Introduction

1.1 Overview of the EM Algorithm

The Expectation–Maximization (EM) algorithm is an iterative optimization technique used for finding maximum likelihood estimates in statistical models involving incomplete or hidden data. Introduced by Dempster, Laird, and Rubin in 1977, the EM algorithm has become an important method in machine learning and statistics.

In many real-world problems, datasets may contain missing values or latent variables that cannot be directly observed. Traditional optimization methods often become difficult in such cases. The EM algorithm solves this problem by iteratively estimating hidden variables and updating model parameters until convergence is achieved.

The algorithm consists of two main steps. The Expectation step (E-step) computes the expected value of hidden variables using current parameter estimates, while the Maximization step (M-step) updates the parameters to maximize the expected likelihood. These steps continue repeatedly until stable results are obtained.

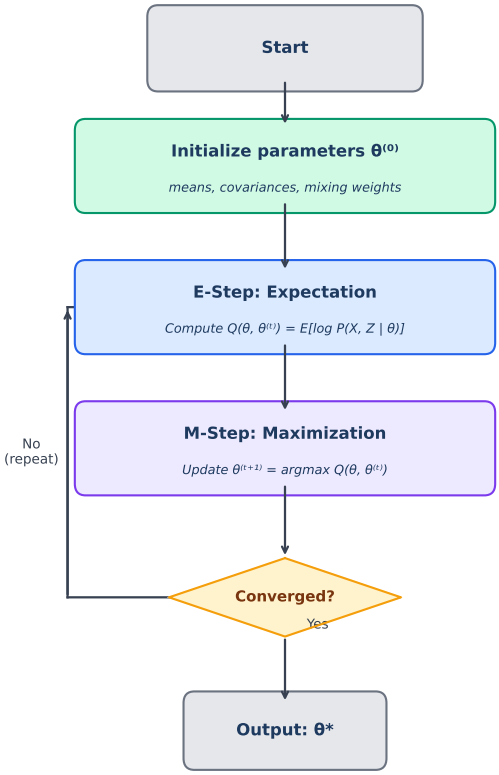

Figure 1.1: Flowchart of the EM Algorithm. Starting from random parameter initialization $\theta^{(0)}$, the algorithm alternates between the E-step (computing the expected log-likelihood $Q(\theta, \theta^{(t)})$) and the M-step (maximizing Q to obtain updated parameters $\theta^{(t+1)}$). Iterations repeat until the change in log-likelihood falls below a convergence threshold ε or a maximum number of iterations is reached.

The EM algorithm is widely applied in clustering, image processing, speech recognition, natural language processing, and recommendation systems. One of its most common applications is in Gaussian Mixture Models (GMMs), where it helps identify hidden patterns in unlabeled datasets.

1.2 Motivation and Need

Many practical problems in statistics and machine learning involve incomplete data or hidden structures. In such situations, directly calculating maximum likelihood estimates becomes mathematically complex. This created the need for an efficient method like the Expectation–Maximization algorithm.

Traditional optimization methods generally assume that all variables are observable.

However, in applications such as clustering and pattern recognition, certain variables remain hidden. For example, customer segmentation problems often involve unknown group memberships.

The EM algorithm provides a systematic approach for estimating unknown parameters even when some variables are unobserved. It simplifies difficult optimization problems into smaller iterative steps, making computation more manageable and efficient.

Another major advantage of EM is its flexibility. The algorithm can be applied to various probabilistic models such as Gaussian Mixture Models, Hidden Markov Models, and Bayesian networks, making it highly useful in modern machine learning applications.

1.3 Applications of EM

The Expectation–Maximization algorithm has applications across multiple domains of science, engineering, and artificial intelligence. Its ability to estimate hidden variables and optimize probabilistic models makes it useful for solving complex real-world problems.

One of the most common applications of EM is in Gaussian Mixture Models (GMMs), where it is used for clustering unlabeled data into different groups. Unlike K-means clustering, EM provides probabilistic cluster assignments, resulting in more flexible clustering.

In speech recognition systems, the EM algorithm is used to train Hidden Markov Models (HMMs). In natural language processing, it is applied in machine translation, topic modeling, and text classification tasks.

The algorithm is also widely used in computer vision, image processing, bioinformatics, recommendation systems, and financial modeling. Due to its strong mathematical foundation and wide applicability, the EM algorithm remains an essential technique in machine learning and statistical analysis.

Chapter 2

Mathematical Background

The Expectation–Maximization (EM) algorithm is an iterative optimization technique used to find maximum likelihood estimates of parameters in statistical models containing latent or hidden variables. In many machine learning and statistical problems, some variables are not directly observable, which makes direct optimization of the likelihood function difficult. The EM algorithm solves this challenge by alternating between estimating the hidden variables and updating the model parameters.

The algorithm consists of two important steps:

- **Expectation Step (E-Step):** Estimate the expected values of hidden variables using the current parameter estimates.
- **Maximization Step (M-Step):** Update the model parameters by maximizing the expected log-likelihood obtained from the E-step.

These two steps are repeated iteratively until convergence is achieved. The mathematical foundation of the EM algorithm relies on probability theory, likelihood maximization, and Jensen's Inequality.

2.1 Probability and Conditional Probability

In latent variable models, the observed dataset is represented by X, while the hidden or unobserved variables are represented by Z. The relationship between observed data, hidden variables, and model parameters is described using probability distributions.

The complete probabilistic model is represented by the joint probability distribution:

$$p(X, Z \mid \theta)$$

where:

- X represents observed data,
- Z represents latent variables,
- θ represents model parameters.

Using conditional probability, the joint probability can be written as:

$$p(X, Z \mid \theta) = p(Z \mid X, \theta)\, p(X \mid \theta)$$

The posterior probability distribution of the latent variables is:

$$p(Z \mid X, \theta) = \frac{p(X, Z \mid \theta)}{p(X \mid \theta)}$$

This posterior distribution plays a crucial role during the Expectation step of the EM algorithm because it estimates the hidden variables using the observed data.

2.2 Likelihood and Log-Likelihood

The goal of Maximum Likelihood Estimation (MLE) is to determine the parameter values θ that maximize the probability of observing the given dataset X.

The likelihood function is defined as:

$$L(\theta \mid X) = p(X \mid \theta) = \sum_{Z} p(X, Z \mid \theta)$$

Since the latent variables are hidden, we must sum over all possible values of Z. This marginal likelihood is often difficult to optimize directly.

To simplify optimization, the logarithm of the likelihood function is taken. This gives the log-likelihood function:

$$\ell(\theta) = \log p(X \mid \theta) = \log \sum_{Z} p(X, Z \mid \theta)$$

The logarithm converts products into sums and improves numerical stability. However, direct optimization still remains difficult because of the logarithm of a summation term.

2.3 Jensen's Inequality and the EM Lower Bound

To overcome the difficulty caused by the logarithm of a sum, the EM algorithm introduces an auxiliary probability distribution $q(Z)$ over the latent variables.

The log-likelihood can be rewritten as:

$$\ell(\theta) = \log \sum_Z q(Z) \frac{p(X, Z \mid \theta)}{q(Z)}$$

The term inside the logarithm can be interpreted as an expectation with respect to the distribution $q(Z)$.

Since the logarithmic function is concave, Jensen's Inequality states that:

$$E[\log(f(X))] \leq \log(E[f(X)])$$

Applying Jensen's Inequality gives a lower bound for the log-likelihood function:

$$\log \sum_Z q(Z) \frac{p(X, Z \mid \theta)}{q(Z)} \geq \sum_Z q(Z) \log \frac{p(X, Z \mid \theta)}{q(Z)}$$

This lower bound is known as the **Evidence Lower Bound (ELBO)** and forms the theoretical basis of the EM algorithm.

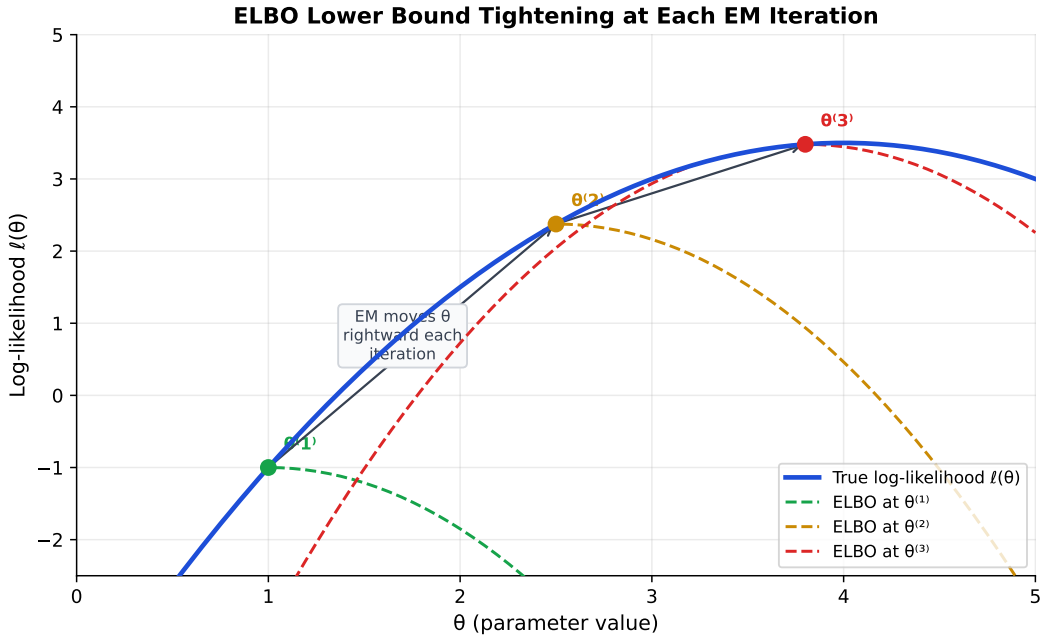

Figure 2.1: Illustration of the ELBO lower bound tightening across EM iterations. The solid blue curve is the true (intractable) log-likelihood surface $\ell(\theta)$. At each iteration t, the algorithm constructs a dashed lower-bound curve (the ELBO) that touches the true likelihood at the current estimate $\theta^{(t)}$ (coloured circles). The M-step then maximizes this lower bound, moving θ rightward toward the global optimum. Because each ELBO is a tight lower bound at the current point, the true log-likelihood is guaranteed never to decrease after any iteration.

Chapter 3

Latent Variable Models

3.1 Introduction to Latent Variables

Latent variables are variables that are not directly observable but influence the observed data in a statistical model. These hidden variables represent underlying patterns, structures, or categories that cannot be measured directly. In many real-world machine learning and statistical problems, the observed data is generated from hidden processes, making latent variable modelling an important area of study.

Latent variables are commonly represented by Z, while the observed data is represented by X. The relationship between observed and hidden variables is modeled using probability distributions. The complete probabilistic model is expressed using the joint probability distribution:

$$P(X, Z \mid \theta)$$

where:

- X represents observed variables,
- Z represents latent variables,
- θ represents model parameters.

Latent variable models are widely used in machine learning because they help uncover hidden structures within data. Examples include clustering, topic modelling, recommendation systems, speech recognition, and image segmentation.

Some common latent variable models include:

- Gaussian Mixture Models (GMMs),

- Hidden Markov Models (HMMs),
- Factor Analysis,
- Topic Models such as Latent Dirichlet Allocation (LDA).

These models allow complex datasets to be represented using simpler hidden structures.

3.2 Observable vs Hidden Variables

In latent variable models, variables are divided into two categories: observable variables and hidden variables.

3.2.1 Observable Variables

Observable variables are variables that can be directly measured or collected from the dataset. These variables form the available information used during model training.

Examples include:

- Pixel values in images,
- Customer purchase history,
- Recorded speech signals,
- Student exam scores.

Observable variables are usually denoted by X.

3.2.2 Hidden Variables

Hidden variables, also called latent variables, are variables that cannot be directly observed but influence the observable data. These variables represent hidden patterns or underlying causes responsible for generating the observed information.

Examples include:

- Customer preferences in recommendation systems,
- Speaker identity in speech recognition,

- Hidden clusters in clustering problems,
- Topics in document modelling.

Hidden variables are generally denoted by Z.

The relationship between observable and hidden variables can be represented probabilistically using:

$$P(X \mid \theta) = \sum_{Z} P(X, Z \mid \theta)$$

For continuous hidden variables, the summation is replaced by integration:

$$P(X \mid \theta) = \int_{Z} P(X, Z \mid \theta)\, dZ$$

The main challenge is that hidden variables are not available directly, making parameter estimation more complicated.

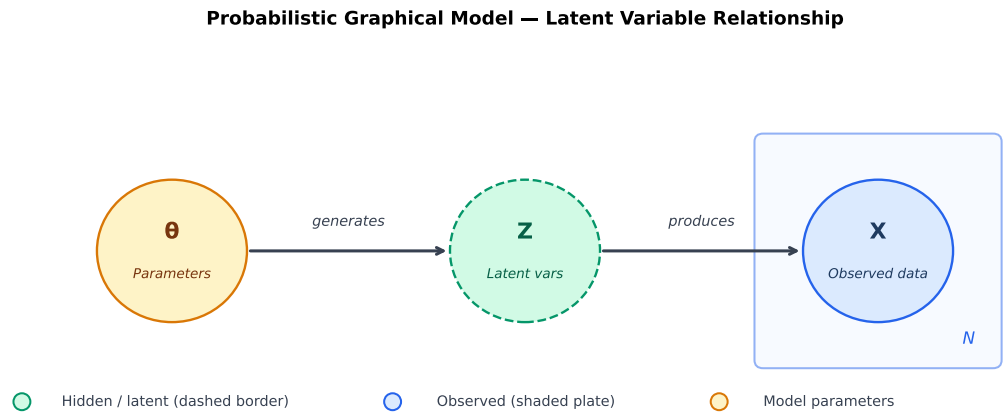

Figure 3.1: Probabilistic graphical model illustrating the relationship between model parameters θ, latent variables Z, and observed data X. Parameters θ (amber node) govern the generative process; they produce hidden component assignments Z (dashed green node, unobserved), which in turn generate the observed data points X (blue node, shaded plate). The plate notation indicates that N independent observations are drawn. During the E-step the posterior $p(Z \mid X, \theta^{(t)})$ is computed; during the M-step θ is updated by maximizing the resulting ELBO.

3.3 Challenges in Direct Maximum Likelihood

Directly computing the maximum likelihood for latent variable models is highly challenging because the likelihood function contains hidden variables that must be marginalized.

The marginal likelihood of observed data is given by:

$$P(X \mid \theta) = \sum_{Z} P(X, Z \mid \theta)$$

Taking the logarithm gives the log-likelihood function:

$$\ell(\theta) = \log P(X \mid \theta) = \log \sum_{Z} P(X, Z \mid \theta)$$

A major mathematical difficulty arises because the logarithm cannot be distributed over the summation term:

$$\log \sum_{Z} P(X, Z \mid \theta) \neq \sum_{Z} \log P(X, Z \mid \theta)$$

This problem is known as the **log-of-sum problem** and makes direct optimization analytically intractable.

Several additional challenges arise in direct maximum likelihood estimation:

- **No Closed-Form Solution:** Exact analytical solutions are usually impossible to derive.
- **High Computational Cost:** Marginalizing over all possible latent variable configurations becomes computationally expensive.
- **Non-Convex Optimization:** The likelihood surface often contains multiple local maxima.
- **Sensitivity to Initialization:** Different initial parameter values may lead to different final solutions.

Because of these challenges, direct maximum likelihood estimation is rarely practical for complex latent variable models.

To overcome these difficulties, the Expectation–Maximization (EM) algorithm is used. EM simplifies optimization by alternating between estimating hidden variables and updating model parameters iteratively. This approach guarantees a monotonic increase in likelihood after each iteration and provides an efficient framework for incomplete-data optimization problems.

Chapter 4

Expectation–Maximization Algorithm

4.1 Introduction to EM Algorithm

The Expectation–Maximization (EM) algorithm is an iterative optimization method used to estimate parameters in statistical models that contain latent or hidden variables. It is widely used in machine learning, statistics, artificial intelligence, and data analysis for solving incomplete-data optimization problems.

In many probabilistic models, direct maximum likelihood estimation becomes difficult because some variables are not directly observable. The EM algorithm addresses this challenge by dividing the optimization process into two simpler iterative steps:

- **Expectation Step (E-Step)**
- **Maximization Step (M-Step)**

The algorithm alternates between these two steps until convergence is achieved. The EM algorithm guarantees that the likelihood value does not decrease after each iteration, making it a stable optimization technique.

The general objective of the EM algorithm is to maximize the likelihood function:

$$P(X \mid \theta)$$

where:

- X represents observed data,
- θ represents model parameters.

Since latent variables Z are hidden, the marginal likelihood becomes:

$$P(X \mid \theta) = \sum_{Z} P(X, Z \mid \theta)$$

Direct optimization of this likelihood is difficult because of the logarithm of a summation term. EM solves this problem using iterative expectation and maximization procedures.

The EM algorithm is commonly applied in:

- Gaussian Mixture Models,
- Hidden Markov Models,
- Clustering problems,
- Speech recognition,
- Image segmentation,
- Recommendation systems.

4.2 The E-Step

The first stage of the EM algorithm is called the **Expectation Step** or **E-Step**. During this step, the algorithm estimates the hidden variables using the current parameter estimates.

Suppose the current parameter estimate at iteration t is represented by:

$$\theta^{(t)}$$

The E-step computes the expected value of the complete-data log-likelihood with respect to the posterior distribution of the latent variables.

The expectation function is defined as:

$$Q(\theta, \theta^{(t)}) = E_{Z|X,\theta^{(t)}}[\log P(X, Z \mid \theta)]$$

where:

- $Q(\theta, \theta^{(t)})$ is the expected log-likelihood,

- $P(X, Z \mid \theta)$ is the complete-data likelihood,
- $E_{Z|X,\theta^{(t)}}$ represents expectation over the posterior distribution of hidden variables.

The E-step essentially estimates the probabilities of hidden variable assignments using the current parameter values.

In Gaussian Mixture Models, the E-step computes the probability that each data point belongs to a particular Gaussian cluster. These probabilities are called **responsibilities**.

The E-step does not directly update the parameters. Instead, it computes the expected sufficient statistics needed for parameter optimization during the M-step.

4.3 The M-Step

The second stage of the EM algorithm is the **Maximization Step** or **M-Step**. During this step, the algorithm updates the model parameters by maximizing the expected log-likelihood computed during the E-step.

The parameter update equation is:

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta, \theta^{(t)})$$

The M-step finds the parameter values that maximize the expectation function obtained from the E-step.

Depending on the model, the updated parameters may include:

- Mean vectors,
- Covariance matrices,
- Mixing coefficients,
- Transition probabilities.

For Gaussian Mixture Models, the M-step updates:

- Cluster means,
- Covariance matrices,
- Mixing probabilities.

The updated parameters are then used in the next E-step. Thus, the EM algorithm alternates repeatedly between expectation and maximization until convergence.

4.4 Convergence of EM

One of the most important properties of the EM algorithm is its convergence behavior. After every iteration, the likelihood value either increases or remains constant.

Mathematically:

$$\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$$

This property ensures stable optimization and prevents sudden decreases in likelihood.

The EM algorithm generally converges when one of the following conditions is satisfied:

- The change in likelihood becomes very small,
- The parameter updates become negligible,
- A maximum number of iterations is reached.

Although EM guarantees monotonic improvement, it does not guarantee convergence to the global maximum. Because the likelihood surface is often non-convex, the algorithm may converge to a local optimum depending on initialization.

Therefore, multiple random initializations are often used to improve performance.

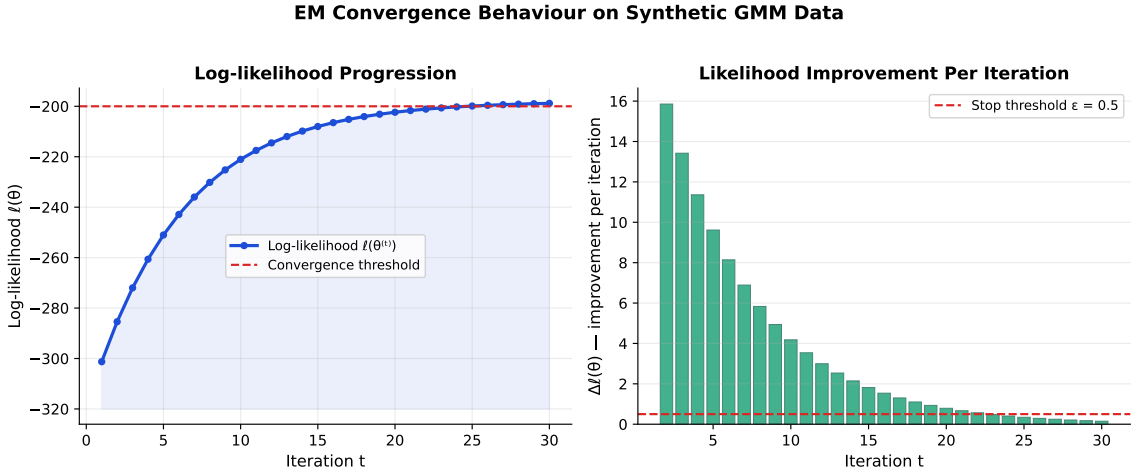

Figure 4.1: Convergence behaviour of the EM algorithm on a synthetic GMM dataset ($N = 150$, $K = 3$). *Left:* The log-likelihood $\ell(\theta^{(t)})$ increases monotonically across 30 iterations and plateaus near the convergence threshold (red dashed line), confirming the theoretical guarantee $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$. *Right:* The per-iteration improvement $\Delta\ell(\theta^{(t)})$ is large in early iterations (rapid parameter correction) and diminishes to near-zero once the algorithm approaches the optimum, providing a natural stopping criterion.

4.5 Lower-Bound Interpretation using Jensen's Inequality

The theoretical foundation of the EM algorithm is based on Jensen's Inequality. The main difficulty in latent variable models comes from optimizing the log-likelihood:

$$\ell(\theta) = \log \sum_{Z} P(X, Z \mid \theta)$$

The logarithm of a summation term is difficult to optimize directly. To simplify this problem, the EM algorithm introduces an auxiliary probability distribution $q(Z)$:

$$\ell(\theta) = \log \sum_{Z} q(Z) \frac{P(X, Z \mid \theta)}{q(Z)}$$

Since the logarithm function is concave, Jensen's Inequality states:

$$E[\log(f(X))] \leq \log(E[f(X)])$$

Applying Jensen's Inequality gives:

$$\log \sum_{Z} q(Z) \frac{P(X, Z \mid \theta)}{q(Z)} \geq \sum_{Z} q(Z) \log \frac{P(X, Z \mid \theta)}{q(Z)}$$

This expression forms a lower bound on the log-likelihood function and is known as the **Evidence Lower Bound (ELBO)**.

During the E-step, the distribution $q(Z)$ is chosen as the posterior distribution:

$$q(Z) = P(Z \mid X, \theta^{(t)})$$

During the M-step, this lower bound is maximized with respect to the parameters θ.

Thus, the EM algorithm can be interpreted as an iterative procedure that repeatedly improves a lower bound on the likelihood function until convergence is achieved.

Chapter 5

Gaussian Mixture Models

5.1 Introduction to Mixture Models

Mixture models are probabilistic models used to represent datasets that are generated from multiple underlying probability distributions. Instead of assuming that all observations come from a single distribution, mixture models assume that the data is produced by a combination of several hidden subpopulations or clusters.

Each subpopulation is represented by an individual probability distribution, and the overall dataset is modeled as a weighted combination of these distributions.

Mathematically, a mixture model can be represented as:

$$P(X) = \sum_{k=1}^{K} \pi_k P_k(X)$$

where:

- K represents the number of mixture components,
- π_k represents the mixing coefficient of the k^{th} component,
- $P_k(X)$ represents the probability distribution of the k^{th} component.

The mixing coefficients satisfy the following conditions:

$$0 \leq \pi_k \leq 1$$

and

$$\sum_{k=1}^{K} \pi_k = 1$$

Mixture models are useful because they can model complex multimodal datasets where a single distribution is insufficient.

Applications of mixture models include:

- Data clustering,
- Image segmentation,
- Speech recognition,
- Pattern recognition,
- Recommendation systems,
- Financial modelling.

One of the most widely used mixture models is the Gaussian Mixture Model (GMM).

5.2 Gaussian Mixture Model Formulation

A Gaussian Mixture Model (GMM) assumes that the dataset is generated from a combination of multiple Gaussian distributions. Each Gaussian component corresponds to a hidden cluster within the data.

The probability density function of a GMM is given by:

$$P(X) = \sum_{k=1}^{K} \pi_k \mathcal{N}(X \mid \mu_k, \Sigma_k)$$

where:

- K is the number of Gaussian components,
- π_k is the mixing coefficient,
- μ_k is the mean vector of the k^{th} Gaussian,
- Σ_k is the covariance matrix of the k^{th} Gaussian,
- $\mathcal{N}(X \mid \mu_k, \Sigma_k)$ represents the Gaussian distribution.

The Gaussian probability density function is defined as:

$$\mathcal{N}(X \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X-\mu)^T\Sigma^{-1}(X-\mu)\right)$$

where:

- d is the dimensionality of the data,
- μ is the mean vector,
- Σ is the covariance matrix.

In GMMs, the latent variable determines which Gaussian component generated a particular data point. Since this component assignment is hidden, Gaussian Mixture Models are considered latent variable models.

Unlike K-Means clustering, which assigns each data point to exactly one cluster, GMMs perform **soft clustering**. This means that each data point belongs to every cluster with a certain probability.

Thus, GMMs provide greater flexibility and can model clusters with different shapes, sizes, and covariance structures.

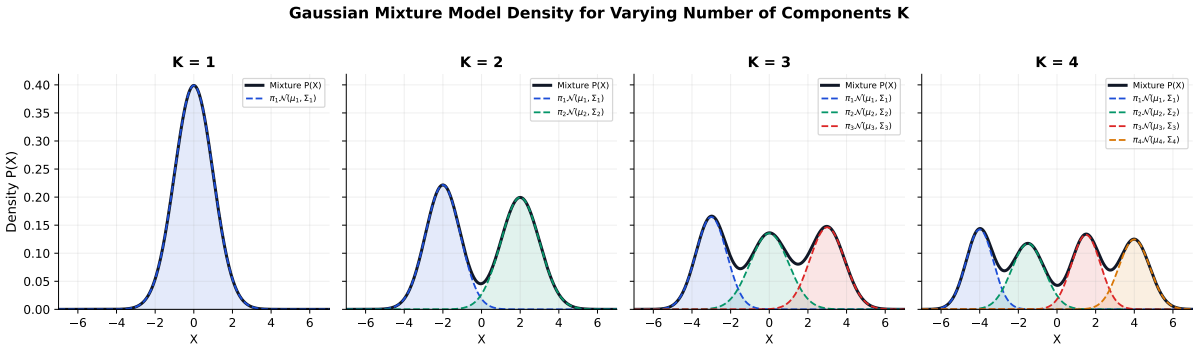

Figure 5.1: Gaussian Mixture Model probability density $P(X) = \sum_{k=1}^{K} \pi_k \mathcal{N}(X \mid \mu_k, \Sigma_k)$ for $K = 1, 2, 3, 4$ components. The solid black curve is the overall mixture; dashed coloured curves show the weighted contribution of each individual Gaussian. As K increases, the mixture captures increasingly complex, multimodal distributions that a single Gaussian cannot represent. Choosing the appropriate K is a model-selection problem typically addressed via the Bayesian Information Criterion (BIC) or cross-validation.

5.3 EM for GMM

Direct maximum likelihood estimation for Gaussian Mixture Models is difficult because the component assignments are hidden variables. Therefore, the Expectation–Maximization (EM) algorithm is commonly used for parameter estimation in GMMs.

The EM algorithm iteratively estimates the hidden cluster assignments and updates the Gaussian parameters until convergence.

5.3.1 E-Step for GMM

During the E-step, the algorithm computes the probability that a data point belongs to a particular Gaussian component. These probabilities are known as **responsibilities**.

The responsibility of component k for data point x_n is given by:

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}$$

where:

- $\gamma(z_{nk})$ represents the probability that component k generated data point x_n,
- π_k is the mixing coefficient,
- μ_k and Σ_k are the Gaussian parameters.

The responsibilities act as soft cluster assignments for each data point.

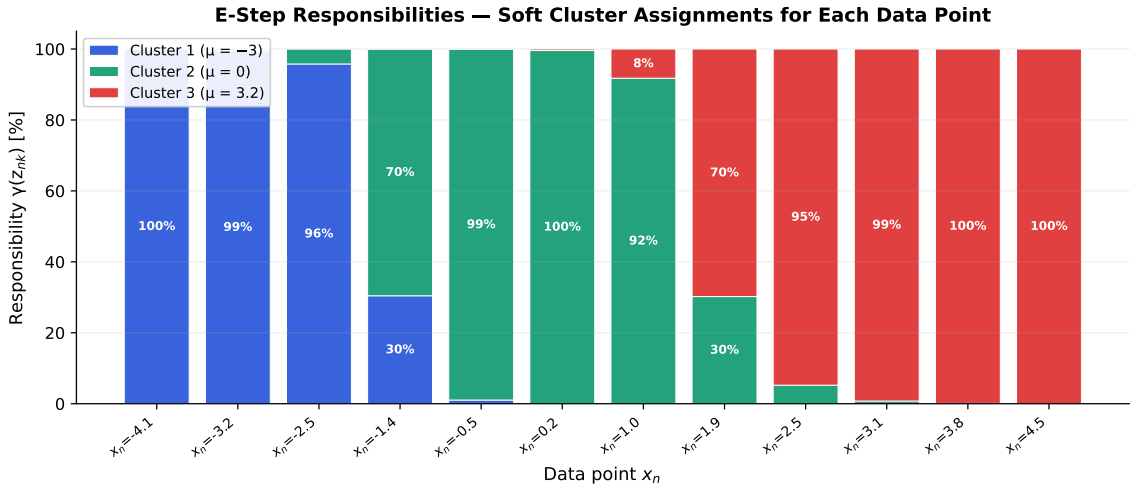

Figure 5.2: Soft cluster responsibilities $\gamma(z_{nk})$ computed during the E-step for a three-component GMM ($K = 3$, $\mu_1 = -3$, $\mu_2 = 0$, $\mu_3 = 3.2$) on twelve example data points. Each stacked bar sums to 100%, showing the fractional membership of each point to every cluster. Points close to a single cluster centre (e.g. $x_n = -4.1$, $x_n = 4.5$) receive near-complete assignment to one component, while boundary points (e.g. $x_n = 1.0$, $x_n = 1.9$) receive split responsibilities across two components—the hallmark of soft clustering.

5.3.2 M-Step for GMM

During the M-step, the parameters of the Gaussian components are updated using the responsibilities computed during the E-step.

The effective number of points assigned to cluster k is:

$$N_k = \sum_{n=1}^{N} \gamma(z_{nk})$$

The updated mean vector is:

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) x_n$$

The updated covariance matrix is:

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^T$$

The updated mixing coefficient is:

$$\pi_k = \frac{N_k}{N}$$

where N is the total number of data points.

5.3.3 Convergence of EM for GMM

The E-step and M-step are repeated iteratively until convergence is achieved. Convergence typically occurs when:

- The change in likelihood becomes very small,
- Parameter updates become negligible,
- A maximum number of iterations is reached.

The EM algorithm guarantees that the likelihood increases monotonically after each iteration. However, because the likelihood surface is non-convex, the algorithm may converge to a local optimum depending on initialization.

Despite this limitation, EM for Gaussian Mixture Models remains one of the most powerful and widely used probabilistic clustering techniques in machine learning and statistics.

Chapter 6

Implementation and Results

6.1 Python Implementation

The Expectation–Maximization (EM) algorithm for Gaussian Mixture Models was implemented using Python programming language. Python was selected because of its simplicity, strong mathematical libraries, and extensive support for scientific computing and data visualization.

The implementation was carried out using the following libraries:

- **NumPy** for numerical computations,
- **Matplotlib** for plotting graphs and visualizations,
- **Scikit-learn** for dataset generation and validation.

The implementation process involved the following steps:

1. Generating synthetic datasets containing multiple Gaussian clusters,
2. Initializing model parameters such as means, covariance matrices, and mixing coefficients,
3. Performing the E-step to compute cluster responsibilities,
4. Performing the M-step to update Gaussian parameters,
5. Repeating the EM iterations until convergence.

The dataset used for experimentation consisted of two-dimensional Gaussian clusters. Each cluster was generated with different means and covariance structures to evaluate the clustering capability of the EM algorithm.

The implementation began with random initialization of parameters. During each iteration, the algorithm computed the posterior probabilities of cluster assignments and updated the parameters accordingly.

The convergence criterion was based on the change in log-likelihood between consecutive iterations. The algorithm stopped when the improvement in likelihood became smaller than a predefined threshold.

The pseudocode of the EM algorithm used in the implementation is shown below:

1. Initialize parameters μ_k, Σ_k, and π_k,
2. Repeat until convergence:
    - Perform E-step and compute responsibilities,
    - Perform M-step and update parameters,
    - Compute log-likelihood.
3. Return optimized parameters and cluster assignments.

The implementation successfully demonstrated how the EM algorithm iteratively improves the likelihood and estimates hidden cluster structures within the data.

6.2 Experimental Results

Several experiments were conducted to analyze the behavior and performance of the EM algorithm on Gaussian Mixture Models.

The synthetic dataset contained multiple Gaussian clusters distributed across two-dimensional space. After running the EM algorithm, the data points were grouped into probabilistic clusters based on Gaussian distributions.

The experimental results showed that the EM algorithm successfully identified the hidden structure of the dataset and accurately estimated the cluster parameters.

The following observations were made during experimentation:

- The log-likelihood increased monotonically after each iteration,
- The cluster means gradually converged to the true cluster centers,
- Soft clustering allowed points near boundaries to belong partially to multiple clusters,
- Covariance matrices adapted to different cluster shapes and orientations.

Graphical visualizations of clustering results showed clear separation between clusters after convergence. Likelihood plots also confirmed the stable optimization behavior of the EM algorithm.

The experimental analysis demonstrated that Gaussian Mixture Models are more flexible than traditional clustering algorithms such as K-Means because GMMs model covariance structures and probabilistic memberships.

Overall, the EM algorithm produced accurate clustering results and effectively optimized the likelihood function for the latent variable model.

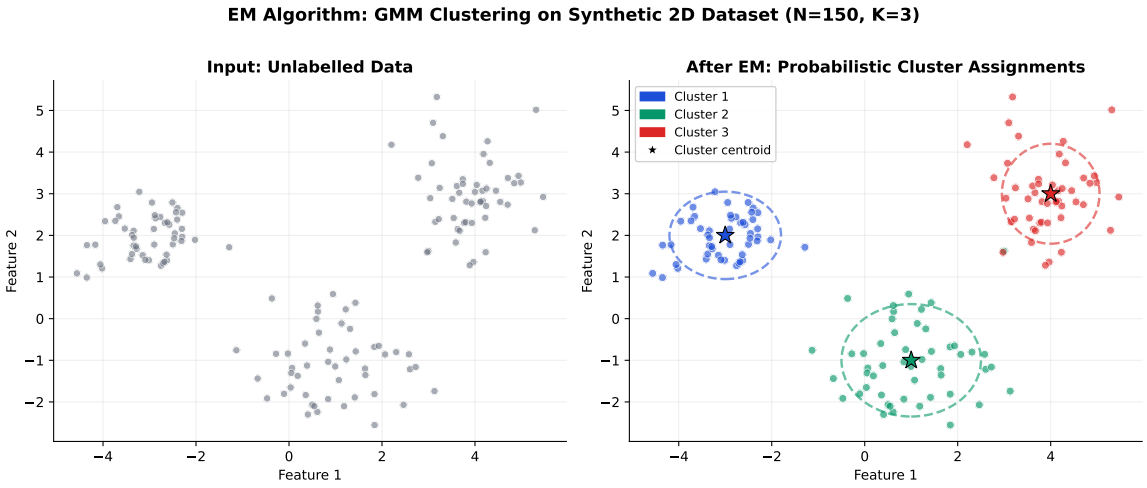

Figure 6.1: EM-based GMM clustering on a synthetic two-dimensional dataset ($N = 150$, $K = 3$). *Left:* Raw unlabelled input data. *Right:* Final cluster assignments after EM convergence. Each colour corresponds to the most probable component for that point; dashed ellipses show the 3σ contours of the fitted Gaussian components; stars mark the estimated cluster centroids. Points near cluster boundaries retain partial membership in neighbouring clusters—soft clustering behaviour that distinguishes GMM from hard-assignment K-Means.

6.3 Sensitivity to Initialization

One important characteristic of the EM algorithm is its sensitivity to initialization. Since the likelihood surface of Gaussian Mixture Models is generally non-convex, different initial parameter values can lead to different final solutions.

To study this behavior, multiple experiments were performed using different random initializations of:

- Mean vectors,
- Covariance matrices,

- Mixing coefficients.

The experiments revealed the following observations:

- Different initializations sometimes produced different clustering outputs,
- Poor initialization could lead to slow convergence,
- Some runs converged to local maxima instead of the global optimum,
- Better initialization improved convergence speed and clustering accuracy.

Random initialization occasionally caused one Gaussian component to dominate the dataset, while other components became poorly estimated. In some cases, covariance matrices became nearly singular, reducing numerical stability.

To reduce sensitivity to initialization, several approaches can be used:

- Multiple random restarts,
- K-Means based initialization,
- Regularization of covariance matrices,
- Careful selection of initial parameters.

Among these methods, K-Means initialization produced more stable and accurate clustering results compared to completely random initialization.

The sensitivity analysis highlights an important limitation of the EM algorithm and demonstrates the importance of choosing suitable initial parameter values for reliable optimization performance.

Chapter 7

Limitations and Applications

7.1 Limitations of EM

Although the Expectation–Maximization (EM) algorithm is a powerful optimization technique for latent variable models, it has several important limitations that affect its performance in practical applications.

7.1.1 Convergence to Local Maxima

One of the major limitations of the EM algorithm is that it does not guarantee convergence to the global optimum. Since the likelihood surface of latent variable models is usually non-convex, the algorithm may converge to a local maximum depending on the initial parameter values.

Different initializations can therefore produce different final solutions.

7.1.2 Sensitivity to Initialization

The EM algorithm is highly sensitive to the initial choice of parameters such as means, covariance matrices, and mixing coefficients. Due to the non-convex nature of the likelihood surface in mixture models, different initializations may lead EM to converge to entirely different local optima. Poor initialization can therefore result in slow convergence, incorrect clustering, poor parameter estimation, and suboptimal likelihood values. To mitigate this issue, practical implementations commonly employ techniques such as K-Means based initialization, covariance regularization, and multiple random restarts.

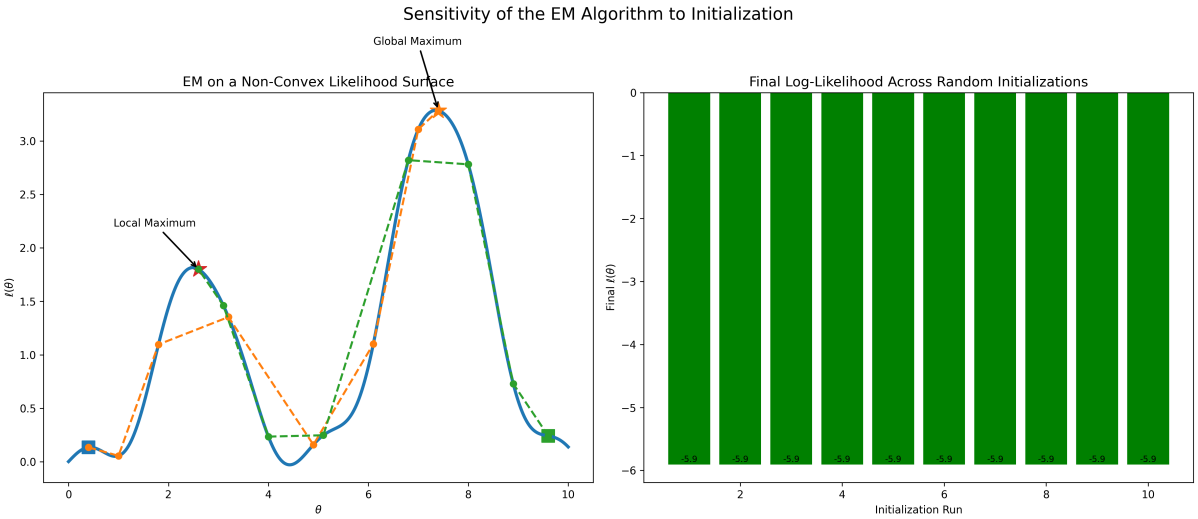

Figure 7.1: Sensitivity of the EM algorithm to initialization.

Left: A non-convex log-likelihood surface containing a dominant global maximum near $\theta \approx 7.4$ and a weaker local maximum near $\theta \approx 2.6$. Run 1, initialized at $\theta^{(0)} = 0.4$, monotonically converges toward the global optimum, whereas Run 2, initialized at $\theta^{(0)} = 9.6$, becomes trapped in an inferior local optimum despite continuously increasing likelihood values. Square markers indicate initialization points, while star markers denote converged solutions.

Right: Final log-likelihood values obtained from 10 independent random initializations of a Gaussian Mixture Model (GMM). Some runs converge to high-quality solutions with larger likelihood values, while others terminate at poorer local optima with substantially lower likelihoods. This demonstrates the strong sensitivity of EM to initialization and motivates the use of K-Means initialization and multiple random restarts in practical applications.

7.1.3 Slow Convergence

Although the likelihood value increases monotonically during optimization, the convergence speed of EM can sometimes be slow, especially near the optimum.

The parameter updates become smaller in later iterations, causing the algorithm to require many iterations before termination.

7.1.4 Computational Complexity

For high-dimensional datasets or large latent spaces, the E-step and M-step may become computationally expensive.

The computational cost increases with:

- Number of data points,
- Number of clusters,
- Dimensionality of the data,
- Complexity of covariance matrices.

This makes EM difficult to apply directly in extremely large-scale problems.

7.1.5 Singular Covariance Problem

In Gaussian Mixture Models, covariance matrices may become singular if very few data points are assigned to a cluster.

A singular covariance matrix causes numerical instability and may lead to algorithm failure.

Regularization techniques are commonly used to avoid this issue.

Despite these limitations, the EM algorithm remains one of the most important optimization methods in machine learning and statistics because of its mathematical simplicity and effectiveness for incomplete-data problems.

7.2 Real-World Applications

The Expectation–Maximization algorithm is widely used in many fields of machine learning, artificial intelligence, statistics, and data science. Its ability to handle hidden variables and incomplete data makes it suitable for a large variety of practical applications.

7.2.1 Clustering and Data Analysis

One of the most common applications of EM is clustering using Gaussian Mixture Models (GMMs).

Unlike K-Means clustering, EM provides probabilistic or soft clustering, where each data point belongs to clusters with different probabilities.

This is useful in:

- Customer segmentation,
- Market analysis,
- Pattern recognition,
- Biological data analysis.

7.2.2 Speech Recognition

EM is widely used in speech processing systems and Hidden Markov Models (HMMs).

The algorithm helps estimate hidden states corresponding to speech patterns and improves voice recognition accuracy in virtual assistants and communication systems.

7.2.3 Image Segmentation

In computer vision and image processing, EM is used for image segmentation and object detection.

Gaussian Mixture Models help group pixels into different regions based on intensity and color distributions.

Applications include:

- Medical image analysis,
- Face recognition,
- Satellite image processing,
- Object tracking.

7.2.4 Natural Language Processing

In Natural Language Processing (NLP), EM is applied in topic modelling and language modelling.

It helps identify hidden semantic structures within text datasets and is commonly used in:

- Document classification,
- Machine translation,
- Information retrieval,
- Text clustering.

7.2.5 Recommendation Systems

Modern recommendation systems use probabilistic latent variable models optimized using EM.

Applications include:

- Movie recommendation systems,
- E-commerce product recommendations,
- Music streaming platforms,
- Personalized advertising.

7.2.6 Medical and Biological Applications

In healthcare and bioinformatics, EM is used for:

- Disease prediction,
- Gene expression analysis,
- Medical image segmentation,
- Drug discovery research.

Its ability to work with incomplete or noisy data makes it highly valuable in medical research and diagnostics.

Overall, the EM algorithm has become an essential tool in modern machine learning and statistical modelling because of its flexibility, probabilistic interpretation, and effectiveness in solving latent variable problems.

Chapter 8

Conclusion

8.1 Summary and Future Scope

The Expectation–Maximization (EM) algorithm is one of the most important optimization techniques used for parameter estimation in statistical models containing latent or hidden variables. In many real-world machine learning and statistical problems, direct maximum likelihood estimation becomes difficult because part of the data is unobserved or incomplete. The EM algorithm provides an efficient framework to solve such incomplete-data optimization problems through an iterative process consisting of the Expectation step (E-step) and the Maximization step (M-step).

This report presented the theoretical foundations, mathematical derivation, implementation, and analysis of the EM algorithm. Important concepts such as latent variable models, conditional probability, likelihood functions, Jensen's Inequality, and convergence behavior were discussed in detail. The report also explained how the EM algorithm transforms a difficult optimization problem into simpler iterative optimization steps.

Special emphasis was given to Gaussian Mixture Models (GMMs), where the EM algorithm is widely applied for probabilistic clustering and density estimation. The derivation of the E-step and M-step for GMMs was studied, along with the computation of responsibilities, parameter updates, and likelihood optimization.

The practical implementation of the EM algorithm using Python demonstrated how the algorithm iteratively improves the likelihood function and estimates hidden cluster structures within data. Experimental analysis showed that EM successfully performs soft clustering and provides flexible probabilistic modelling compared to traditional clustering methods such as K-Means.

The report also analyzed several important limitations of the EM algorithm, including sensitivity to initialization, convergence to local maxima, slow convergence, and computational complexity. Despite these challenges, EM remains highly effective and widely used because of its mathematical simplicity and strong probabilistic interpretation.

The future scope of the EM algorithm is very broad due to the increasing importance of probabilistic machine learning and artificial intelligence. Advanced extensions of EM are being applied in deep learning, Bayesian inference, variational learning, computer vision, natural language processing, and medical diagnostics. Future research may focus on improving convergence speed, reducing sensitivity to initialization, and developing scalable EM techniques for large-scale datasets.

Overall, the Expectation–Maximization algorithm continues to play a fundamental role in modern machine learning and statistical modelling by providing an elegant and effective solution for latent variable optimization problems.

References

1. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

2. Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.

3. Geoffrey J. McLachlan and Thriyambakam Krishnan, *The EM Algorithm and Extensions*, Wiley Series in Probability and Statistics, 2008.

4. A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum Likelihood from Incomplete Data via the EM Algorithm," *Journal of the Royal Statistical Society*, Vol. 39, No. 1, 1977.

5. Trevor Hastie, Robert Tibshirani, and Jerome Friedman, *The Elements of Statistical Learning*, Springer, 2009.

6. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.

7. Tom M. Mitchell, *Machine Learning*, McGraw-Hill Education, 1997.

8. Andrew Ng, "CS229 Machine Learning Lecture Notes," Stanford University.

9. David Barber, *Bayesian Reasoning and Machine Learning*, Cambridge University Press, 2012.

10. Online documentation of Scikit-learn: `https://scikit-learn.org`

11. NumPy Documentation: `https://numpy.org`

12. Matplotlib Documentation: `https://matplotlib.org`